

German University in Cairo
Faculty of Engineering and Materials Science
Mechatronics Engineering

MCTR 1021
Optimization Techniques for Multi-Cooperative Systems

Cooperative UAV Placement for Disaster-Zone Coverage

Milestone 3: Simulated Annealing
Progress Report

Team 47

Ahmed Adel	58-1051
Mohamed Ayman	58-1182
Adham Ashraf	58-1398
Ahmed Elshamy	58-25496
Karim Ahmed	58-6418
Loay Mohamed	58-0300

Course team: Ahmed Mostafa

18 October 2026

Abstract

This milestone implements simulated annealing (SA) for the three-dimensional placement of a fixed UAV fleet serving weighted ground targets. The inherited geometric model maximizes weighted unique coverage minus a hovering-power cost, subject to flight-volume bounds, minimum pairwise separation and multi-hop connectivity to a ground control station. The implementation uses only the Python standard library; MATLAB visualizes the saved results.

A feasible constructive initializer is followed by a mixture of single-UAV three-dimensional and altitude-only moves. Initial-temperature calibration, geometric cooling and all complete candidate evaluations share an explicit budget. A frozen study contains 105 runs and 400,000 optimizer evaluations: main comparisons at three sizes, a two-parameter sensitivity study, and clustered and separated target cases. Initial placements are paired by seed with a uniform random-search baseline.

SA achieves mean weighted coverage of 98.81% and 99.84% on the medium and large cases, respectively, while using less power than random search. It wins all ten paired runs on each of those cases, but loses on mean fitness in the small case. Larger moves improve the five-seed medium-case sensitivity results. All final runs retain feasible incumbents. Validation includes 43 automated tests, agreement with 63 historical model evaluations, an exhaustive restricted-grid benchmark with optimal fitness -16 reached in ten out of ten SA runs, and audits of exported results. The benchmark certifies only its finite domain; the study does not establish continuous global optimality or real-world radio performance.

Contents

Abstract **i**

1 Introduction **1**

1.1 Motivation and problem 1

1.2 Milestone objectives and scope 1

2 Background and Related Work **2**

2.1 UAV-assisted wireless networks 2

2.2 Simulated annealing 2

2.3 Position of the present study 2

3 Problem Formulation **3**

3.1 Variables and parameters 3

3.2 Coverage and power 3

3.3 Feasibility constraints 4

3.4 Scalar fitness 4

4 Simulated Annealing Implementation **5**

4.1 Feasible initialization and neighborhood 5

4.2 Temperature calibration and acceptance 5

4.3 Accounting, software and baseline 6

5 Experimental Design **7**

5.1 Instances and shared settings 7

5.2 Fairness and reproducibility 7

5.3 Interpretation of figures and statistics 8

6 Validation **9**

6.1 Model and implementation checks 9

6.2 Exact finite-grid benchmark 9

6.3 Artifact and frozen-study audits 9

7 Results and Discussion **11**

7.1 Main comparison 11

7.2 Deployment geometry and convergence 15

7.3 Sensitivity to temperature and move scale 17

7.4 Clustered and separated demand 19

7.5 Feasibility and computational behavior 21

7.6 Limits of the evidence 22

8 Conclusion and Future Work **23**

A Reproducibility and Artifact Guide **24**

B Additional Deployment and Convergence Figures **26**

B.1 Small case 26

B.2 Large case 28

B.3 Clustered case 30

B.4 Separated case 32

References **34**

List of Figures

7.1 Small case: SA and random-search mean fitness (with sample SD), weighted coverage, power per UAV and runtime over ten seeds. 12

7.2 Medium case: SA and random-search mean fitness (with sample SD), weighted coverage, power per UAV and runtime over ten seeds. 13

7.3 Large case: SA and random-search mean fitness (with sample SD), weighted coverage, power per UAV and runtime over ten seeds. 14

7.4 Medium case, selected best SA run (seed 101): original feasible initialization. This is the starting placement of the selected run, not an average over seeds. 15

7.5 Medium case, seed 101: best feasible placement found within 4,000 evaluations. The two panels show footprints and the three-dimensional connected fleet. 16

7.6 Medium case, seed 101: current and best-feasible fitness and calibrated geometric cooling. Initialization and calibration have no plotted search temperature. 16

7.7 Medium sensitivity study: five seeds per setting; error bars are sample SD. Temperature variations keep scale at 0.05, and scale variations keep the temperature multiplier at 1. 18

7.8 Clustered targets: mean fitness with sample SD, weighted coverage, power per UAV and runtime over five seeds per method. 19

7.9 Separated targets: mean fitness with sample SD, weighted coverage, power per UAV and runtime over five seeds per method. 20

7.10 Recorded mean runtime and weighted coverage by fleet size. Target counts and budgets also change, so the plot does not establish an isolated complexity or scaling law. 21

B.1 Small case: original feasible placement of the best SA run, with its seed identified in the supplied plot. 26

B.2 Small case: best feasible placement of the best SA run, with its seed identified in the supplied plot. 27

B.3 Small case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot. 27

B.4 Large case: original feasible placement of the best SA run, with its seed identified in the supplied plot. 28

B.5 Large case: best feasible placement of the best SA run, with its seed identified in the supplied plot. 28

B.6 Large case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot. 29

B.7 Clustered case: original feasible placement of the best SA run, with its seed identified in the supplied plot. 30

B.8 Clustered case: best feasible placement of the best SA run, with its seed identified in the supplied plot. 30

B.9 Clustered case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot. 31

B.10 Separated case: original feasible placement of the best SA run, with its seed identified in the supplied plot. 32

B.11 Separated case: best feasible placement of the best SA run, with its seed identified in the supplied plot. 32

B.12 Separated case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot. 33

List of Tables

3.1 Model notation and units. 3

5.1 Final experiment scope. Budgets count complete evaluations per run. 7

7.1 Main comparison over ten seeds per method and case. 11

7.2 Paired fitness differences using identical case/seed initial placements. 11

7.3 Initial and best fitness of the selected illustrative SA runs. These are selected examples, not group means. 15

7.4 Medium-case sensitivity over five paired seeds per setting. 17

7.5 Challenging-case comparison over five seeds per method and case. 19

A.1 Computational source and artifact map. 25

Chapter 1

Introduction

1.1 Motivation and problem

Cooperative UAVs can provide temporary aerial service when terrestrial infrastructure is disrupted. Their placement must balance service coverage with resource consumption while keeping the fleet connected. This report studies that tradeoff through a static geometric optimization problem, not a complete wireless-network simulator.

The decision is the three-dimensional location of each UAV. Ground targets have fixed coordinates and positive priority weights. A target is served when it lies within at least one geometric antenna footprint. UAVs must remain within the prescribed volume, maintain separation, and reach the ground control station (GCS) through direct or multi-hop distance-based links.

1.2 Milestone objectives and scope

Milestone 3 adds a reproducible SA solver to the Milestone 2 objective and constraints. Its deliverables are the algorithm, illustrative solutions, validation, and analysis over different problem sizes and parameter settings. This report documents the actual implementation, the final experimental protocol, the generated MATLAB figures and the observed strengths and limitations.

The scope is deliberately explicit: there is no interference, fading, SINR, terrain obstruction, moving demand, battery state, trajectory optimization or physical flight-energy simulation. Hovering power is an assumed linear surrogate measured in watts. Coverage and connectivity are geometric. These assumptions permit an interpretable comparison while restricting the conclusions that can be drawn about deployment in practice.

The final study is version `final_v1`, algorithm version 1.1.0. Earlier exploratory runs are excluded from all reported aggregate statistics. Main results are means over repeated seeded runs, not only favorable individual examples.

Chapter 2

Background and Related Work

2.1 UAV-assisted wireless networks

Mozaffari et al. [1] survey UAV applications, challenges and tradeoffs in wireless networks. Their tutorial provides context for aerial infrastructure and the interaction between placement and communication requirements. It is used here as background, not as a numerical benchmark for the present geometric objective. The simplified footprint and range constraints in this project do not reproduce a full propagation or resource-allocation model.

2.2 Simulated annealing

Kirkpatrick, Gelatt and Vecchi [2] describe optimization through simulated annealing. The central mechanism allows occasional unfavorable changes at positive temperature and reduces their acceptance as cooling proceeds. In this project fitness is maximized, so a feasible worsening move with change $\Delta F < 0$ is accepted with probability $\exp(\Delta F/T)$. This creates a controlled means of exploring beyond immediately improving moves; it does not guarantee an optimum under a finite evaluation budget.

2.3 Position of the present study

The contribution is a transparent application of SA to the inherited UAV model, with explicit feasibility, budget accounting, paired initialization, saved artifacts and sensitivity analysis. Uniform whole-fleet random search provides a reproducible baseline. A custom exhaustive finite-grid version of the same model provides an exact reference value. No numerical reproduction of a published UAV benchmark, superiority over literature, or general claim about all UAV-placement methods is made. A matched hill-climbing baseline would be needed to isolate the benefit of annealing from that of local search itself.

Chapter 3

Problem Formulation

3.1 Variables and parameters

Let N UAVs have positions $\mathbf{p}_i = (x_i, y_i, z_i)$ and let M ground targets have coordinates $\mathbf{t}_m = (x_m^T, y_m^T)$ and weights $w_m > 0$. The optimization variables are the $3N$ position coordinates. Coverage flags and communication edges are derived from these coordinates; they are not independently optimized variables.

Table 3.1: Model notation and units.

Symbol	Definition
$\mathbf{p}_i, \mathbf{g}$	UAV and GCS coordinates (m)
w_m	Dimensionless priority of ground target m
θ	Antenna half-beam angle (degrees)
$R_c, d_{\min}$	Communication radius and minimum UAV separation (m)
P_0, β	Base power (W) and altitude coefficient (W/m)
C, P, V	Weighted unique coverage, total power (W), aggregate violation
α, λ	Power-cost and violation-penalty coefficients

3.2 Coverage and power

The footprint radius is $r_i = z_i \tan\theta$. A target contributes its weight once, including when several UAVs cover it:

$$b_m(\mathbf{P}) = \mathbb{1}\left\{\exists i : \sqrt{(x_i - x_m^T)^2 + (y_i - y_m^T)^2} \le z_i \tan\theta\right\}, \tag{3.1}$$

$$C(\mathbf{P}) = \sum_{m=1}^{M} w_m b_m(\mathbf{P}), \qquad C_{\text{frac}} = \frac{C}{\sum_m w_m}, \tag{3.2}$$

$$P(\mathbf{P}) = \sum_{i=1}^{N} (P_0 + \beta z_i). \tag{3.3}$$

Weighted coverage fraction differs from the unweighted fraction of targets served. The assumed linear power model is inherited from Milestone 2; no empirical aerodynamic calibration is claimed. Power is not energy in joules. With $\theta = 45°$, raising altitude enlarges a footprint but also increases its power cost.

3.3 Feasibility constraints

For each UAV and coordinate $a \in \{x, y, z\}$, the bounds satisfy $l_a \leq p_{ia} \leq u_a$. Every UAV pair must satisfy $\|\mathbf{p}_i - \mathbf{p}_j\|_2 \geq d_{\min}$. Construct an undirected graph containing the UAVs and GCS; an edge exists exactly when their three-dimensional distance is at most R_c. Every UAV must be reachable from the GCS. Multi-hop paths are allowed, and all eligible edges are recorded.

The implemented violation terms are

$$V_b = \sum_{i,a} \left[\max(0, l_a - p_{ia}) + \max(0, p_{ia} - u_a)\right], \tag{3.4}$$

$$V_s = \sum_{i<j} \max(0, d_{\min} - \|\mathbf{p}_i - \mathbf{p}_j\|_2), \tag{3.5}$$

$$V_c = \#\{i : i \text{ is not reachable from the GCS}\}, \qquad V = V_b + V_s + V_c. \tag{3.6}$$

This inherited sum mixes distance deficits and a disconnected-node count. It is a numerical penalty construction, not a physical quantity with a single unit. Feasibility requires $V = 0$.

3.4 Scalar fitness

The unchanged Milestone 2 fitness is

$$\boxed{F(\mathbf{P}) = C(\mathbf{P}) - 0.1P(\mathbf{P}) - 10000V(\mathbf{P}).} \tag{3.7}$$

Final-study SA rejects infeasible candidates. Reported best solutions therefore have $F = C - 0.1P$. A negative fitness is valid and does not indicate infeasibility. The coefficient 0.1 expresses a preference between coverage and power; this experiment does not construct a Pareto frontier. Raw fitness should be compared within a case, since total priority and fleet size differ between cases.

Chapter 4

Simulated Annealing Implementation

4.1 Feasible initialization and neighborhood

The initializer builds a connected fleet incrementally. It chooses an anchor uniformly from the GCS and already placed UAVs, samples coordinates within a communication-radius box around that anchor, and checks volume bounds, anchor distance and separation. Accepted points therefore extend a connected graph. Fixed coordinate axes are handled directly. The construction cap is 10,000 partial proposals; partial fleets restart before attempts divisible by 500. Construction failure is reported explicitly and does not prove infeasibility of the problem.

A complete initial fleet is scored once. Scalar bounds and pair-distance checks during construction are counted separately from full objective evaluations. SA and random search use the same seed and initializer, giving paired starting placements for each case/seed.

Each SA candidate selects one UAV and uses a 3D translation with probability 0.8 or altitude-only translation with probability 0.2. For each moved axis, the offset is uniform in $[-s(u_a - l_a), s(u_a - l_a)]$, with baseline $s = 0.05$. Coordinates crossing a boundary are reflected back into the interval; they are not clipped or resampled. Altitude-only moves cannot change the horizontal layout. Coordinated multi-UAV moves are not implemented.

4.2 Temperature calibration and acceptance

Up to 100 calibration candidates are sampled around the unchanged initial placement. At least one search evaluation is reserved when the budget permits. Let L contain strictly positive fitness losses from feasible calibration candidates. Then

$$T_0 = m_T \begin{cases} \overline{L}/[-\log(0.8)], & L \neq \varnothing, \\ 1, & L = \varnothing, \end{cases} \tag{4.1}$$

where $m_T = 1$ in the baseline. The target 0.8 is a heuristic used in calibration, not an observed or guaranteed acceptance rate. Calibration candidates can improve the best-feasible archive without changing the initial current state.

For K search evaluations, candidate index $j = 0, \ldots, K-1$ uses

$$T_j = T_0(0.01)^{j/(K-1)} \quad (K > 1). \tag{4.2}$$

For $K = 1$, use T_0. Cooling advances with every evaluated search candidate, including rejected and infeasible candidates. For a feasible candidate with $\Delta F = F_{\text{candidate}} - F_{\text{current}}$,

$$\Pr(\text{accept}) = \begin{cases} 1, & \Delta F \geq 0, \\ \exp(\Delta F / T_j), & \Delta F < 0. \end{cases} \tag{4.3}$$

Infeasible candidates are rejected. Best-feasible updates require strict improvement, and the current, final and best-feasible states are stored separately.

Algorithm 1 Budgeted feasible simulated annealing

Require: Problem, budget B, integer seed, scale s, multiplier m_T
1: Construct an initial feasible placement X; report failure if construction fails.
2: Evaluate X through the budget controller; initialize best archive $X^* = X$.
3: Evaluate up to 100 neighbors of unchanged X, reserving a search evaluation.
4: Update X^* from feasible calibration candidates and compute T_0.
5: Set K to the remaining number of full evaluations.
6: **for** $j = 0$ to $K - 1$ **do**
7: Compute T_j; choose an operator and UAV; propose reflected candidate Y.
8: Evaluate Y through the budget controller, even if infeasible.
9: **if** Y is feasible **then**
10: **if** $F(Y) > F(X^*)$ **then**
11: $X^* \leftarrow Y$.
12: **end if**
13: $\Delta \leftarrow F(Y) - F(X)$.
14: **if** $\Delta \geq 0$ or $U < \exp(\Delta / T_j)$, $U \sim \mathcal{U}(0, 1)$ **then**
15: $X \leftarrow Y$.
16: **end if**
17: **end if**
18: Record sampled trace and aggregate counters.
19: **end for**
20: **return** X^* and its metrics, with final state X recorded separately.

4.3 Accounting, software and baseline

One budget controller counts initialization scoring, calibration and search. A completed run consumes exactly its assigned budget. A budget exhausted by initialization is marked insufficient for search rather than successful. All evaluated infeasible candidates count, whereas cheap construction checks and post-run validation do not consume the optimizer budget. The trace is sampled every 25 evaluations plus initialization, first search and the final evaluation; aggregate counters include every candidate.

Uniform random search keeps the same feasible initial incumbent and spends each remaining evaluation on an independently sampled whole fleet, uniform in the coordinate bounds. It performs no free rejection sampling and no calibration. Thus budgets are equal in full evaluations, although proposal distributions and feasible-candidate rates differ.

Computation uses Python 3.9+ standard-library code, executed with site packages disabled. The recorded experiment runtime used Python 3.13.2. The evaluator costs approximately $O(NM + N^2)$ per placement for coverage and graph construction. The graph reachability traversal follows construction of its eligible edges. MATLAB reads exported CSV/JSON files to generate figures; it does not rerun the optimizer.

Chapter 5

Experimental Design

5.1 Instances and shared settings

The five final instances use $x, y \in [0, 500]$ m, $z \in [30, 120]$ m, GCS $(250, 250, 0)$ m, $R_c = 200$ m, $d_{\min} = 15$ m, $\theta = 45°$, $P_0 = 100$ W and $\beta = 0.5$ W/m. Target weights are generated uniformly in $[1, 10]$. Small, medium and large target coordinates are uniform in the ground square, generated with seeds 101, 102 and 103, respectively.

Clustered targets alternate between centers $(150, 150)$ and $(350, 350)$ m with Gaussian coordinate standard deviation 35 m (generation seed 104). Separated targets use centers $(70, 70)$ and $(430, 430)$ m with standard deviation 20 m (seed 105). Gaussian coordinates are clipped to the square. Both cases have eight UAVs and 50 targets. The exact generated coordinates and weights are frozen in the case files; generation seeds are distinct in role from optimizer seeds.

Table 5.1: Final experiment scope. Budgets count complete evaluations per run.

Study/case	UAVs	Targets	Budget	Runs	Optimizer seeds
Main: small	4	10	2,000	20	100–109
Main: medium	8	50	4,000	20	100–109
Main: large	16	150	5,000	20	100–109
Sensitivity: medium	8	50	4,000	25	200–204
Challenging: clustered	8	50	4,000	10	300–304
Challenging: separated	8	50	4,000	10	300–304

The main and challenging studies compare feasible SA with random search. Sensitivity uses five one-factor-at-a-time configurations: baseline $(m_T, s) = (1, 0.05)$; temperature multipliers 0.5 and 2 at fixed scale; and scales 0.025 and 0.10 at fixed multiplier. All other settings and the medium target layout remain unchanged. Sensitivity outcomes were not used to revise the frozen main-study settings.

5.2 Fairness and reproducibility

A six-run, 1,000-evaluation pilot preceded the freeze, with measured solver runtimes approximately 0.106–0.765 s. Proposed budgets were retained. Each final run has an independent `random.Random(seed)` generator, exact saved configuration and paired initialization where methods are compared. Source and case hashes are checked before and after runs. The final scope is 60 main, 25 sensitivity and 20 challenging runs, totaling 105 runs and 400,000 full optimizer evaluations.

Reported solver runtime includes construction, calibration and search, but excludes result serialization and MATLAB plotting. Mean fitness, weighted coverage and power describe successful feasible incumbents. Sample standard deviation uses denominator $n - 1$. Failed runs would remain in runtime and failure-rate summaries rather than quality means; no final run failed.

5.3 Interpretation of figures and statistics

The supplied MATLAB figures visualize the frozen results. Deployment and convergence examples select the highest-fitness SA run for each case; sensitivity examples select the best baseline run. They are explicitly labeled best-run examples and are not average trajectories. Aggregate comparison figures and tables use every seed. Error bars show sample standard deviations, not confidence intervals. Single-case seeds measure optimizer variability on that fixed layout, not uncertainty over arbitrary disaster zones.

Chapter 6

Validation

6.1 Model and implementation checks

The test suite contains 43 tests and passes under Python with third-party site packages disabled. Coverage includes overlap without double counting, geometric boundary behavior, bounds and separation, GCS reachability, invalid inputs, reflection, acceptance, seed repeatability, temperature calibration and evaluation-index cooling. It also checks output round trips, budget edge cases and frozen-input tampering. The evaluator matches 63 recorded Milestone 2 reference placements. These checks support preservation of the inherited model; they are not empirical validation of radio or aerodynamic assumptions.

6.2 Exact finite-grid benchmark

An exactly solvable restricted version of the same model uses two UAVs and four allowed sites:

$$\mathcal{S} = \{(0, 0, 10), (20, 0, 10), (0, 0, 20), (20, 0, 20)\}. \tag{6.1}$$

Targets are $(0, 0)$ and $(20, 0)$ with weights 2 and 3, respectively. The GCS is $(0, 0, 0)$, bounds are $x \in [0, 20]$, $y = 0$, $z \in [10, 20]$, communication radius is 40 m and minimum separation is 5 m. Beam angle and power parameters retain their defaults. Exhaustive enumeration considers $4^2 = 16$ ordered placements, 12 feasible. The exact optimal fitness is -16.

For example, the fleet at $(0, 0, 10)$ and $(20, 0, 10)$ covers both targets, consumes 210 W, and has $F = 5 - 0.1(210) = -16$. Since both UAVs must have altitude at least 10 m, no feasible fleet can use less than 210 W, and coverage cannot exceed 5. This establishes the bound $F \leq -16$, attained by that placement.

The benchmark SA uses the same finite domain, starts at the two altitude-20 sites and changes one UAV to a sampled allowed site. All ten seeds (0–9), each with a 500-evaluation budget, reach the optimum, giving zero absolute optimality gap. Oracle enumeration uses 16 evaluations; SA uses 5,000 across its ten runs. These costs are separate from the final study. This is a custom exact benchmark, not a reproduced published benchmark. It does not certify optimality on the larger continuous instances.

6.3 Artifact and frozen-study audits

The final audit verifies all 105 runs, all 15 summary rows and 400,000 recorded optimization evaluations. It checks exact cases, configurations, seeds, source hashes, phase totals, initial and

best placements, objective values, coverage flags, communication edges, aliases and the sampled cooling schedule. Each complete audit adds 210 evaluation calls for initial/best recomputation outside the optimizer budgets. Auditing reuses the production evaluator, so it establishes artifact consistency rather than independent correctness of every model equation.

All final runs retained feasible solutions. This means that bounds, separation and GCS reachability hold for their saved incumbents. It does not mean that every candidate was feasible or that initialization will always succeed on other instances. The feasible initializer and archive are important reasons both methods can return feasible solutions despite rejected proposals.

Chapter 7

Results and Discussion

7.1 Main comparison

Table 7.1 summarizes the three instance sizes. Higher fitness and coverage are preferred; lower power is preferred. Coverage is weighted coverage fraction. Averages and standard deviations describe ten runs per method, and runtimes are the recorded experiment timings.

Table 7.1: Main comparison over ten seeds per method and case.

Case	Method/setting	Fitness mean ± SD	Coverage	W/UAV	Time (s)
Small	SA	-16.888 ± 9.538	59.40%	122.60	0.130
Small	Random	-15.557 ± 2.374	74.50%	139.70	0.127
Medium	SA	178.865 ± 7.983	98.81%	136.67	0.759
Medium	Random	124.096 ± 7.428	82.83%	146.89	0.838
Large	SA	643.536 ± 5.222	99.84%	136.06	3.225
Large	Random	574.064 ± 12.612	93.07%	142.96	3.276

On the small case, SA's lower mean power (122.60 rather than 139.70 W/UAV) does not compensate for its lower weighted coverage (59.40% rather than 74.50%). Random search therefore has the better mean fitness, -15.557 versus -16.888. SA also has greater variability (SD 9.538 versus 2.374). Local trapping and initialization sensitivity are plausible interpretations, but were not isolated experimentally. The result should not be described as uniform superiority of SA.

On the medium case, SA raises weighted coverage from 82.83% to 98.81% while reducing power from 146.89 to 136.67 W/UAV. Mean fitness improves by 54.770. On the large case, coverage rises from 93.07% to 99.84%, power falls from 142.96 to 136.06 W/UAV, and fitness improves by 69.472. SA wins all ten paired seeds in each of these cases.

Table 7.2: Paired fitness differences using identical case/seed initial placements.

Case	Mean $F_{SA} - F_{random}$	SA wins
Small	-1.331	4/10
Medium	54.770	10/10
Large	69.472	10/10
Clustered	8.632	5/5
Separated	9.326	5/5

These are descriptive paired results; no statistical significance claim is made. Raw fitness across case sizes is not directly comparable because fleet size and total target priority change.

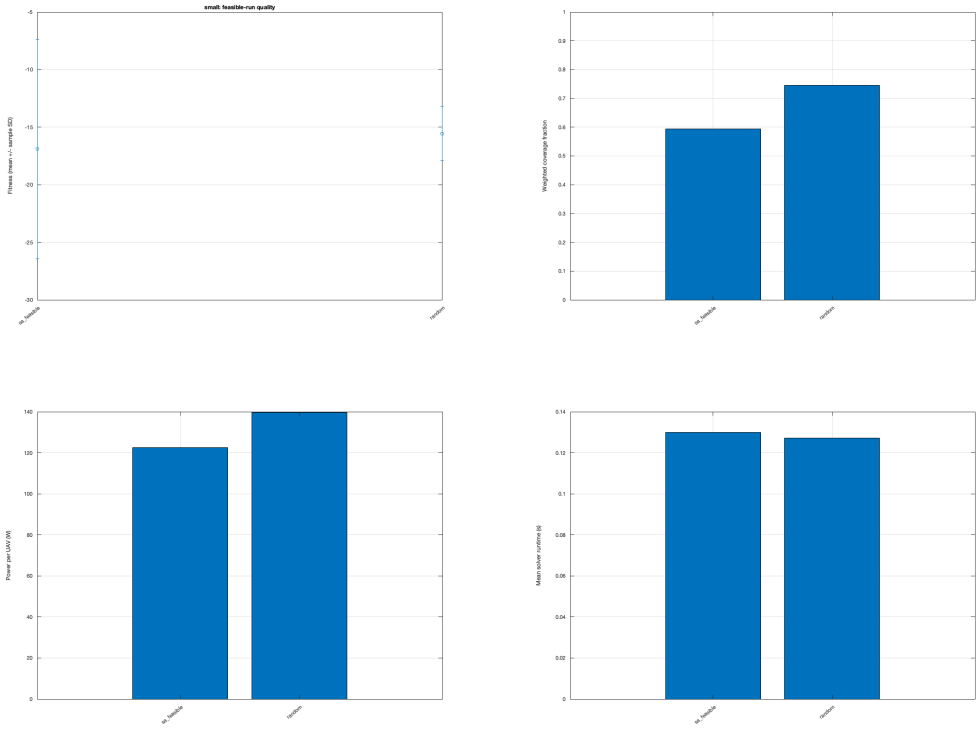


Figure 7.1: Small case: SA and random-search mean fitness (with sample SD), weighted coverage, power per UAV and runtime over ten seeds.

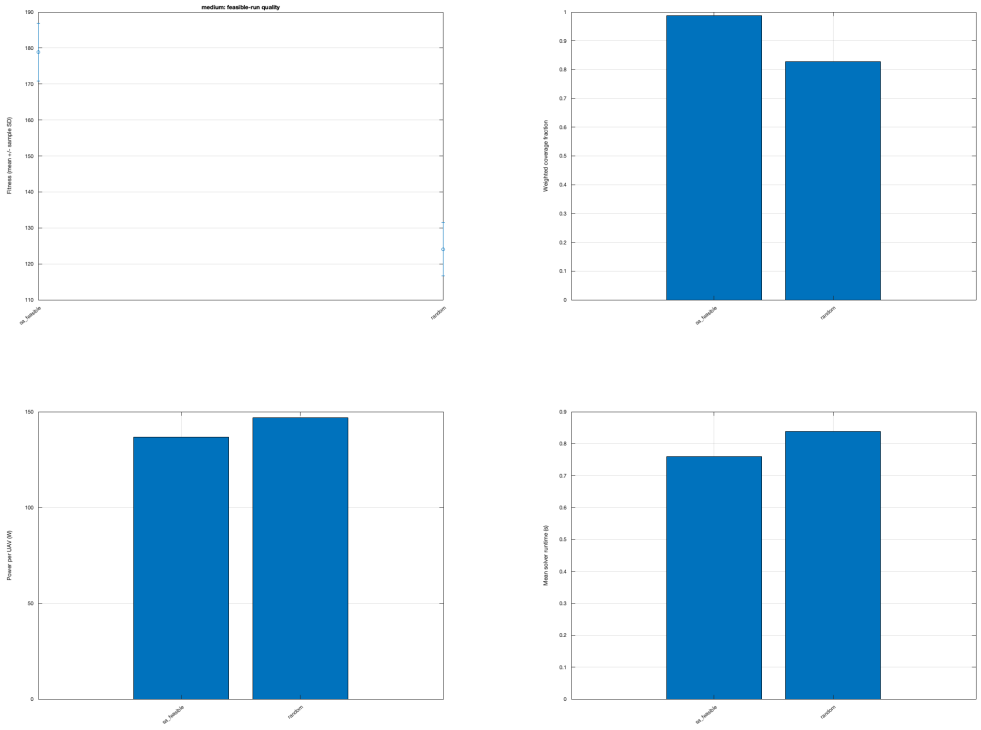


Figure 7.2: Medium case: SA and random-search mean fitness (with sample SD), weighted coverage, power per UAV and runtime over ten seeds.

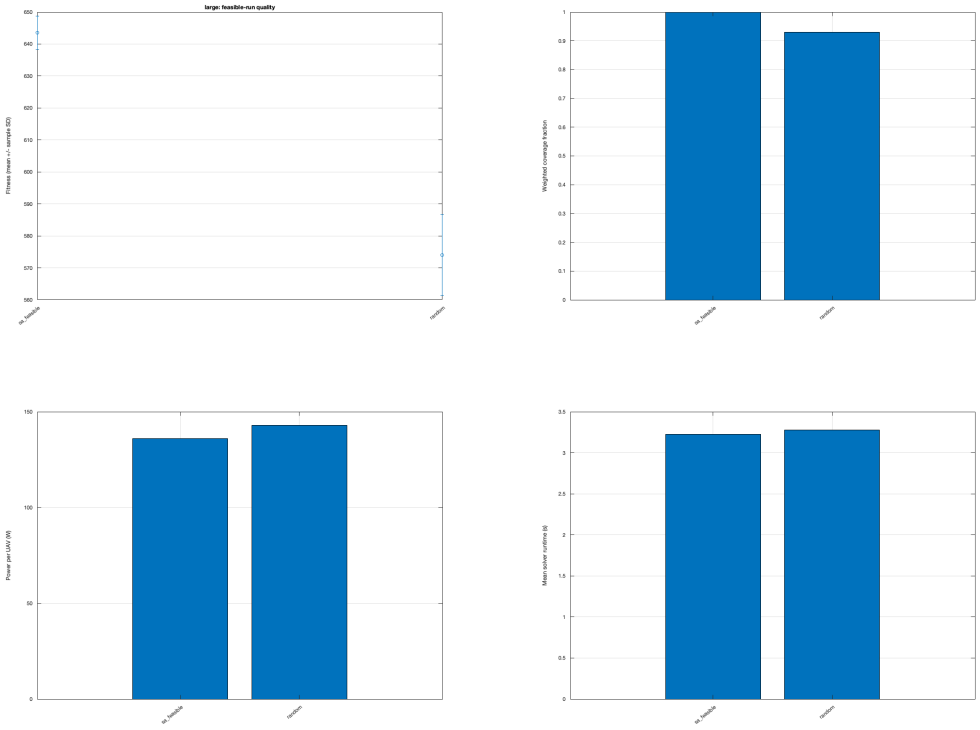


Figure 7.3: Large case: SA and random-search mean fitness (with sample SD), weighted coverage, power per UAV and runtime over ten seeds.

7.2 Deployment geometry and convergence

Figures 7.4 and 7.5 compare the recorded initial and best placements of the selected medium-case run. Green targets are covered, gray targets are uncovered, marker area encodes priority, blue triangles denote UAVs and the red square is the GCS. Dotted circles are coverage footprints. Gray lines denote all distance-eligible communication links, not a selected routing tree; a UAV can reach the GCS indirectly. Altitude labels explain the differing footprint sizes.

Table 7.3: Initial and best fitness of the selected illustrative SA runs. These are selected examples, not group means.

Case	Seed	Initial fitness	Best fitness
Small	107	−29.774	−3.112
Medium	101	36.626	185.208
Large	105	267.635	650.062
Clustered	303	116.935	149.055
Separated	302	−106.203	171.830

Table 7.3 quantifies the improvements behind the illustrated deployments. In particular, the favorable small-case example must not be used to replace the ten-run result: its best fitness is much higher than the small-case SA mean, which still trails random search.

Figure 7.6 shows rapid early gains followed by smaller improvements for this selected run. The current fitness can decrease when a worsening feasible move is accepted, whereas the best-feasible archive is nondecreasing. The straight cooling line on a logarithmic temperature axis reflects geometric cooling. A late plateau within the budget does not establish a global optimum. Additional case deployments and traces are provided in Appendix B.

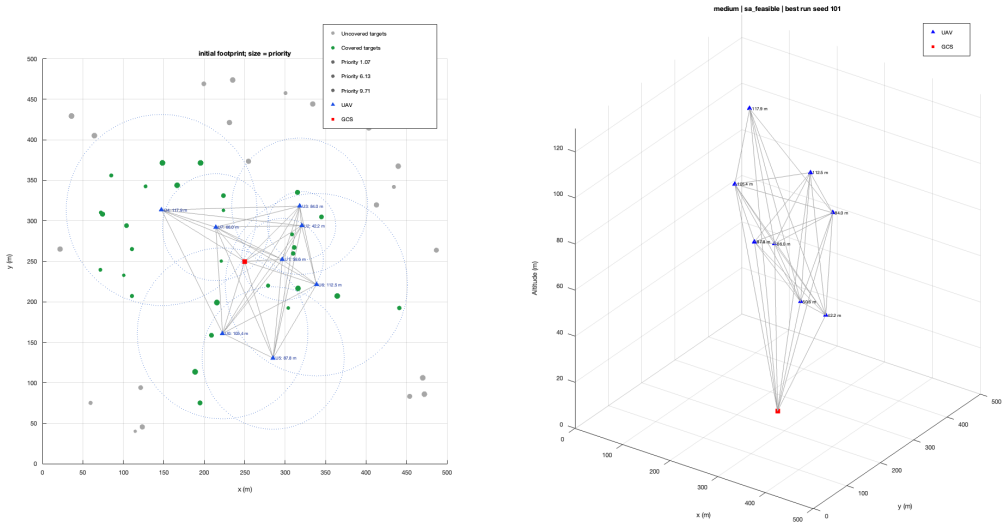


Figure 7.4: Medium case, selected best SA run (seed 101): original feasible initialization. This is the starting placement of the selected run, not an average over seeds.

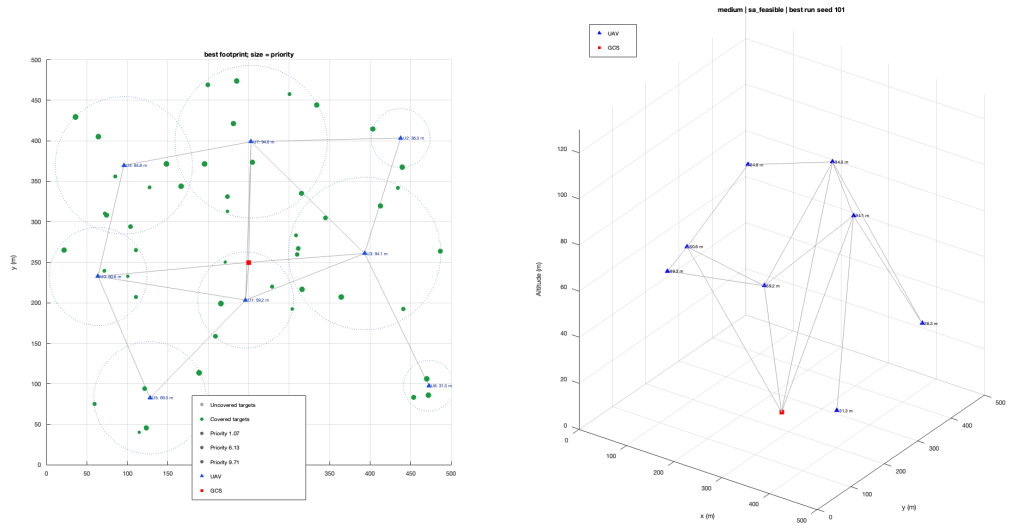


Figure 7.5: Medium case, seed 101: best feasible placement found within 4,000 evaluations. The two panels show footprints and the three-dimensional connected fleet.

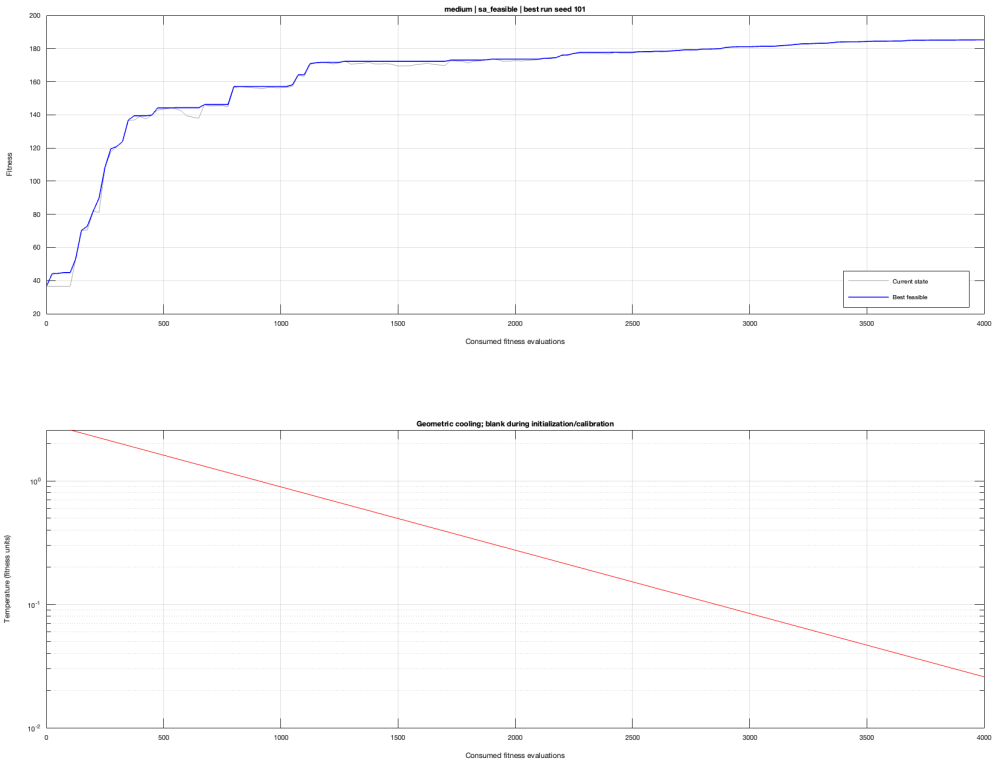


Figure 7.6: Medium case, seed 101: current and best-feasible fitness and calibrated geometric cooling. Initialization and calibration have no plotted search temperature.

7.3 Sensitivity to temperature and move scale

Table 7.4 uses five seeds on the same medium instance. Its baseline differs from the main mean because the seed set differs. Halving the initial-temperature multiplier reduces mean fitness from 172.946 to 151.420; doubling it increases mean fitness to 177.547. This is consistent with a benefit from exploration under the tested budget, but does not establish a universal temperature choice.

Table 7.4: Medium-case sensitivity over five paired seeds per setting.

Case	Method/setting	Fitness mean ± SD	Coverage	W/UAV	Time (s)
Medium	Baseline	172.946 ± 12.808	96.04%	134.00	0.758
Medium	$T_0 \times 0.5$	151.420 ± 30.714	87.73%	130.59	0.659
Medium	$T_0 \times 2$	177.547 ± 12.573	97.85%	134.85	0.690
Medium	$s = 0.025$	140.634 ± 37.719	83.74%	129.53	0.687
Medium	$s = 0.10$	182.789 ± 1.619	100.00%	136.12	0.629

Halving move scale to 0.025 gives the lowest mean fitness, 140.634, and largest SD, 37.719. Doubling scale to 0.10 gives the highest mean, 182.789, with SD 1.619 and 100% coverage in all five runs. Larger moves explore more broadly but produce fewer feasible proposals: 93.35% compared with 99.26% for scale 0.025. A high feasible-proposal rate alone is therefore not sufficient for good optimization.

This one-factor-at-a-time design does not test temperature–scale interactions or identify globally optimal parameters. The observed settings were not used to retune the frozen main results. Figure 7.7 shows the full comparison.

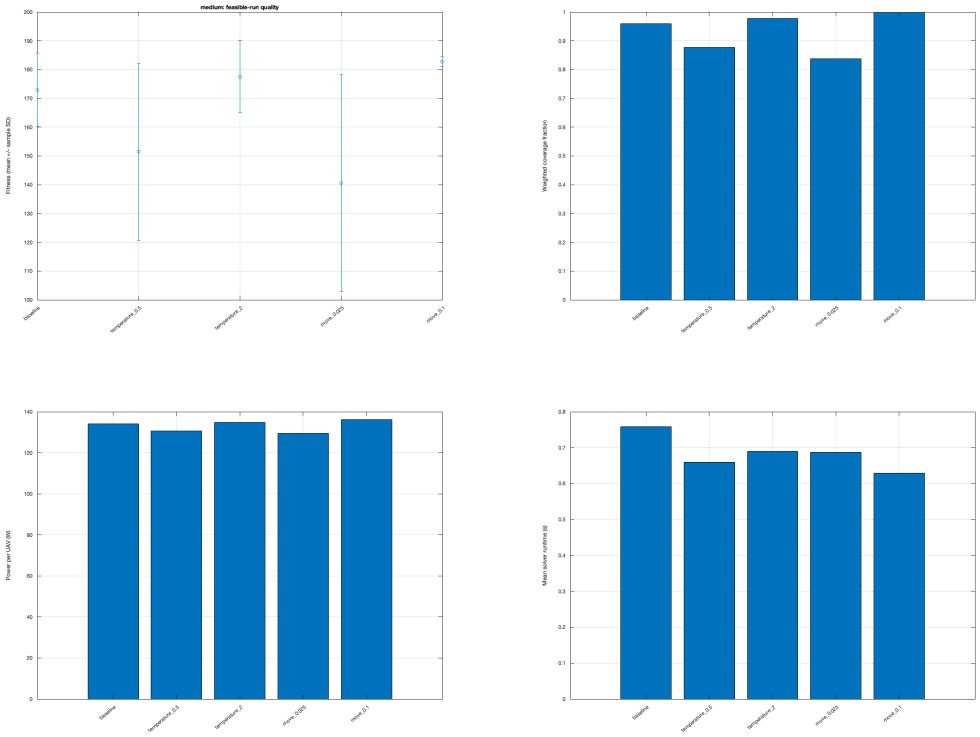


Figure 7.7: Medium sensitivity study: five seeds per setting; error bars are sample SD. Temperature variations keep scale at 0.05, and scale variations keep the temperature multiplier at 1.

7.4 Clustered and separated demand

Both methods achieve 100% weighted coverage on clustered targets. SA improves mean fitness by 8.632 through reducing power from 133.09 to 122.30 W/UAV. The clustered geometry allows concentrated coverage with lower-altitude footprints while preserving links.

Table 7.5: Challenging-case comparison over five seeds per method and case.

Case	Method/setting	Fitness mean ± SD	Coverage	W/UAV	Time (s)
Clustered	SA	147.204 ± 1.238	100.00%	122.30	0.602
Clustered	Random	138.572 ± 1.907	100.00%	133.09	0.630
Separated	SA	168.783 ± 3.553	99.38%	119.01	0.622
Separated	Random	159.457 ± 1.954	99.91%	132.41	0.654

On separated targets, random search has slightly higher mean coverage (99.91% versus 99.38%), but SA lowers power from 132.41 to 119.01 W/UAV. The power saving outweighs the coverage difference under the fixed scalar objective. SA wins all five paired seeds on both challenging instances. This illustrates why fitness, coverage and power must be reported together: the best scalar score need not have maximum coverage alone. Figures 7.8 and 7.9 show these tradeoffs.

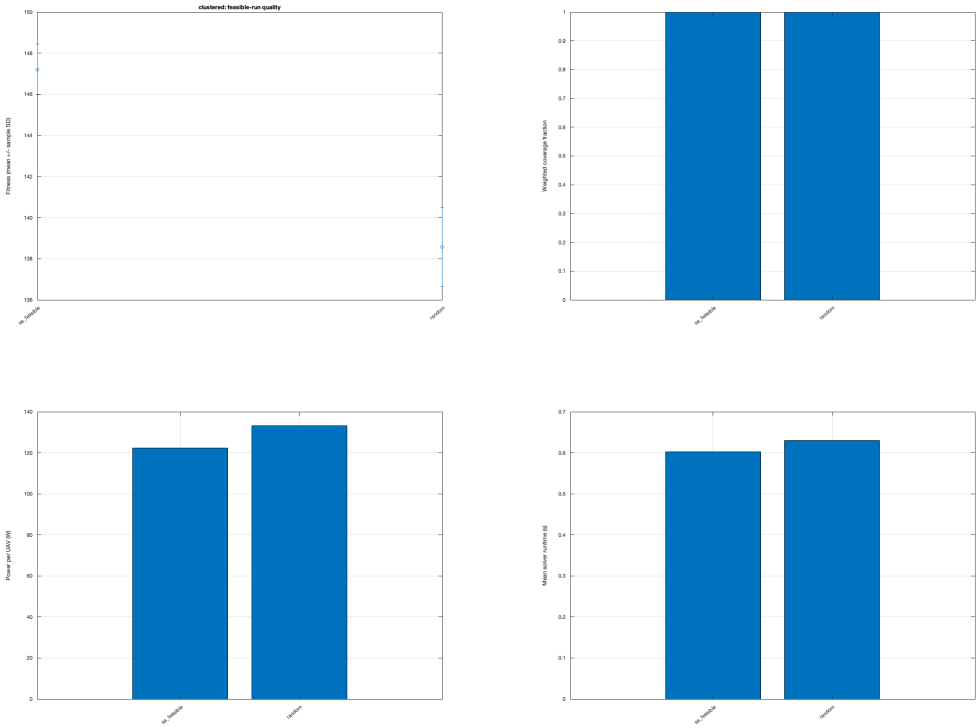


Figure 7.8: Clustered targets: mean fitness with sample SD, weighted coverage, power per UAV and runtime over five seeds per method.

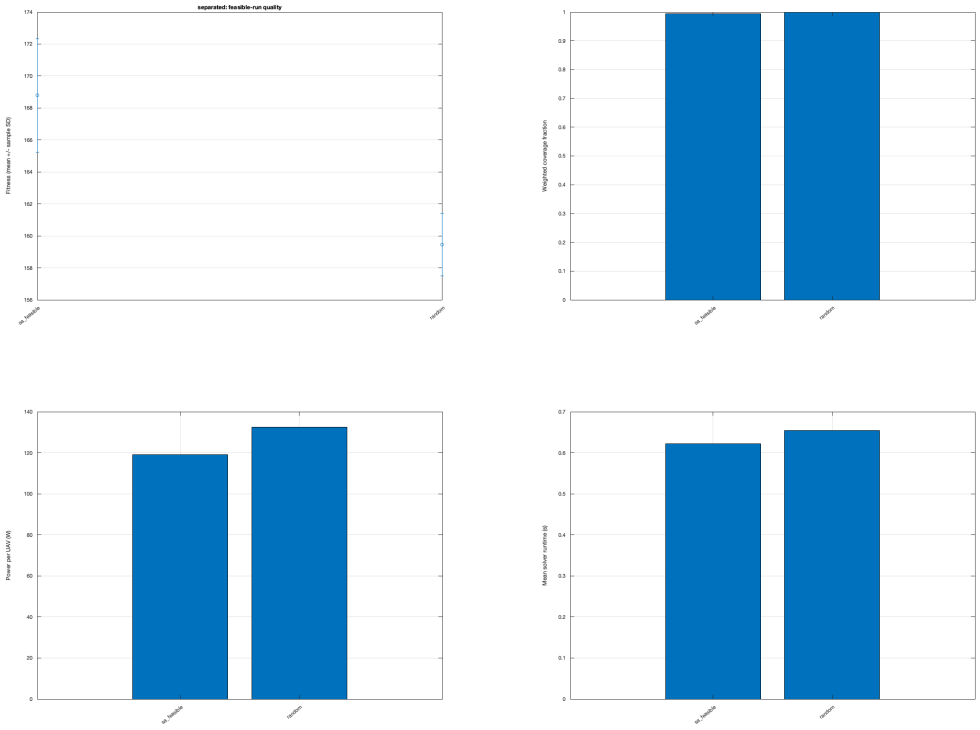


Figure 7.9: Separated targets: mean fitness with sample SD, weighted coverage, power per UAV and runtime over five seeds per method.

7.5 Feasibility and computational behavior

SA's feasible search-proposal rates are 94.60%, 96.48% and 99.43% for small, medium and large cases. Random search yields 25.59%, 35.75% and 71.60%. Local moves often preserve a feasible connected fleet, whereas independent whole-fleet draws more often violate constraints. This explains an important difference in search opportunity; the experiment does not isolate the contribution of annealing from that of the neighborhood and feasibility handling.

Mean SA solver runtime increases from 0.130 s on small to 0.759 s on medium and 3.225 s on large. Random search takes 0.127, 0.838 and 3.276 s, respectively. These values are machine- and load-dependent. Figure 7.10 is descriptive, not a controlled scaling experiment: UAV count, target count and evaluation budget all change together. Total recorded runtime across the 105 final runs is 113.208 s, excluding exports and plotting.

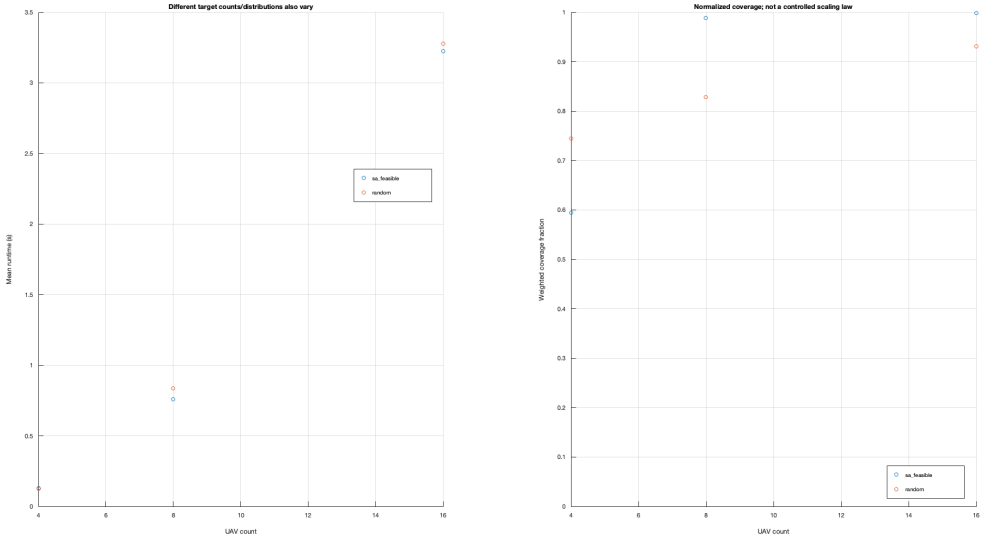


Figure 7.10: Recorded mean runtime and weighted coverage by fleet size. Target counts and budgets also change, so the plot does not establish an isolated complexity or scaling law.

7.6 Limits of the evidence

Each case uses one frozen target layout. Repeated seeds quantify solver variability on that layout rather than generalization to new layouts. There is no confidence interval for the paired advantage, matched hill-climbing or restart baseline, operator ablation, or controlled budget sweep. The sensitivity study uses only five seeds and does not vary the objective coefficient, radio range or separation constraint. The tiny exact benchmark supports correctness on its restricted domain, not large-instance optimality. Finally, deterministic geometry and a linear hovering-power surrogate do not establish field performance under interference, obstacles or moving users.

Chapter 8

Conclusion and Future Work

This milestone delivers a standard-library SA implementation for the inherited cooperative UAV-placement model, a frozen and audited 105-run study, exact finite-domain validation, and MATLAB visualizations. Explicit counting includes initialization scoring and temperature calibration within each optimization budget. Best-feasible solutions remain separate from current and final states.

SA improves both weighted coverage and power consumption relative to uniform random search on the medium and large instances, winning every paired seed there. It also improves fitness on clustered and separated cases. However, it loses on average on the small case, where its coverage reduction outweighs its power saving. Larger moves are beneficial in the tested medium sensitivity study; this observation is instance-specific and does not retroactively change the main protocol.

The most useful extensions are a matched hill-climbing baseline, multiple independently generated layouts per case, paired uncertainty intervals, and a fixed-budget comparison of restarts or larger moves on the small instance. A small factorial temperature–scale study would address parameter interactions. More demanding exact or bounded instances would strengthen optimality comparisons. Physical extensions should separately evaluate interference, propagation uncertainty, movement costs and battery dynamics rather than interpreting the current geometric surrogate as a complete communication model.

Appendix A

Reproducibility and Artifact Guide

The computational project is supplied separately from this report source. Commands below are run from the computational project root. New output directories are required to preserve previous results.

```
python -B -S -m unittest discover -s tests -v
python -B -S -m milestone3.final_experiments verify
python -B -S -m milestone3 demo --seed 0 --budget 2000
    --output results/new-demo
```

The last command is shown on two lines for readability; enter it as one command. A fresh frozen reproduction can be prepared with:

```
python -B -S -m milestone3 pilot --budget 1000 --seeds 0
    --output results/reproduction/pilot
python -B -S -m milestone3.final_experiments freeze
    --version reproduction_v1
    --config experiments/reproduction_v1.json
    --pilot results/reproduction/pilot
python -B -S -m milestone3.final_experiments run
    --config experiments/reproduction_v1.json
```

Each logical command above should be entered on one line. MATLAB figures are generated from exported data using `run_matlab_figures.m`. The supplied PNG figures are incorporated here. Base MATLAB is used for visualization, and no third-party Python package is required for the computation or tests.

Each run includes configuration and metrics JSON, initial and final placement CSVs, eligible edge CSVs, targets and convergence. Here "final placement" means best feasible, not the last visited state. Compatibility aliases preserve the plotting interface. The final frozen configuration SHA-256 is:

```
bd2973c0702000b2db26fd1b7c50fcda6a2
ea7edd41ef29b4fedddb7f84dd07a
```

The two lines form one 64-character hexadecimal digest. The final audit covers 60 main runs (220,000 evaluations), 25 sensitivity runs (100,000) and 20 challenging runs (80,000). Pilot, benchmark and post-run audit evaluations are accounted for separately.

Table A.1: Computational source and artifact map.

Path	Purpose
`milestone3/model.py`	Geometry, fitness and constraints
`milestone3/search.py`	Initializer, budgeted SA and random search
`milestone3/experiments.py`	Runs and sample statistics
`milestone3/final_experiments.py`	Freeze, execution guards and final audit
`milestone3/validation.py`	Saved-artifact consistency checks
`cases/`	Serialized target layouts and physical parameters
`results/final_v1/`	Main, sensitivity and challenging runs
`results/final_v1_benchmark/`	Exact-grid benchmark results
`tests/`	Automated model, solver and artifact checks
`matlab/`	Plotting and import checks

Appendix B

Additional Deployment and Convergence Figures

These supplementary figures show selected best SA runs, not average behavior. Initial and best panels for each case refer to the same seed. All numerical conclusions use the aggregate tables in the results chapter.

B.1 Small case

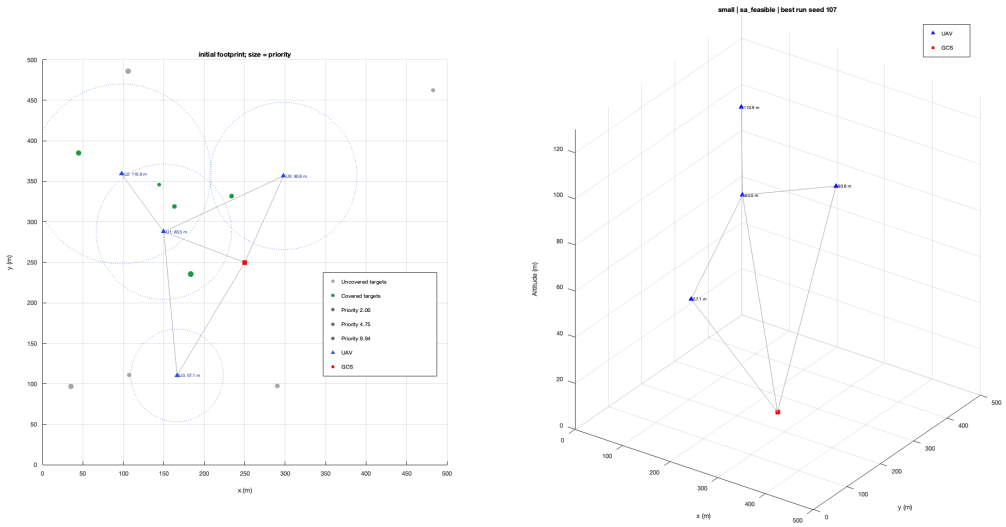


Figure B.1: Small case: original feasible placement of the best SA run, with its seed identified in the supplied plot.

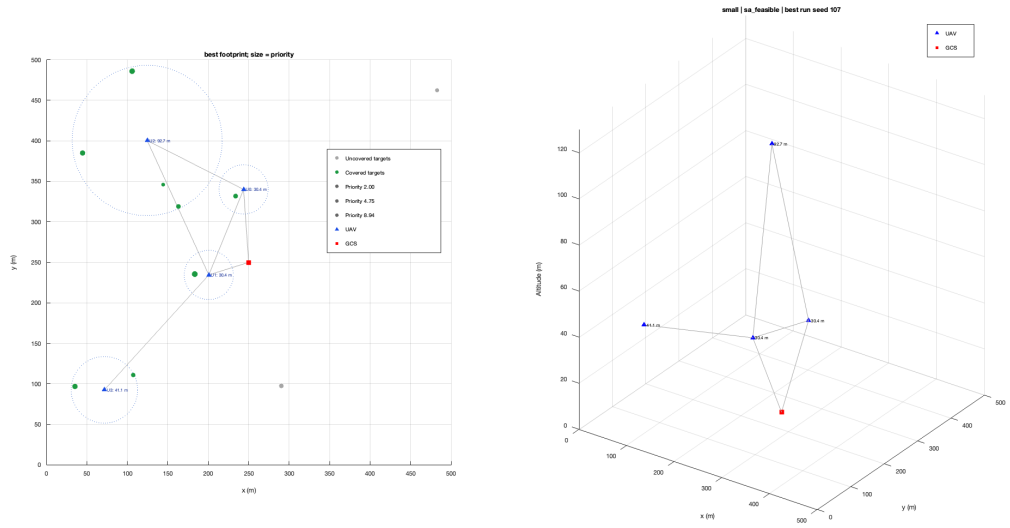


Figure B.2: Small case: best feasible placement of the best SA run, with its seed identified in the supplied plot.

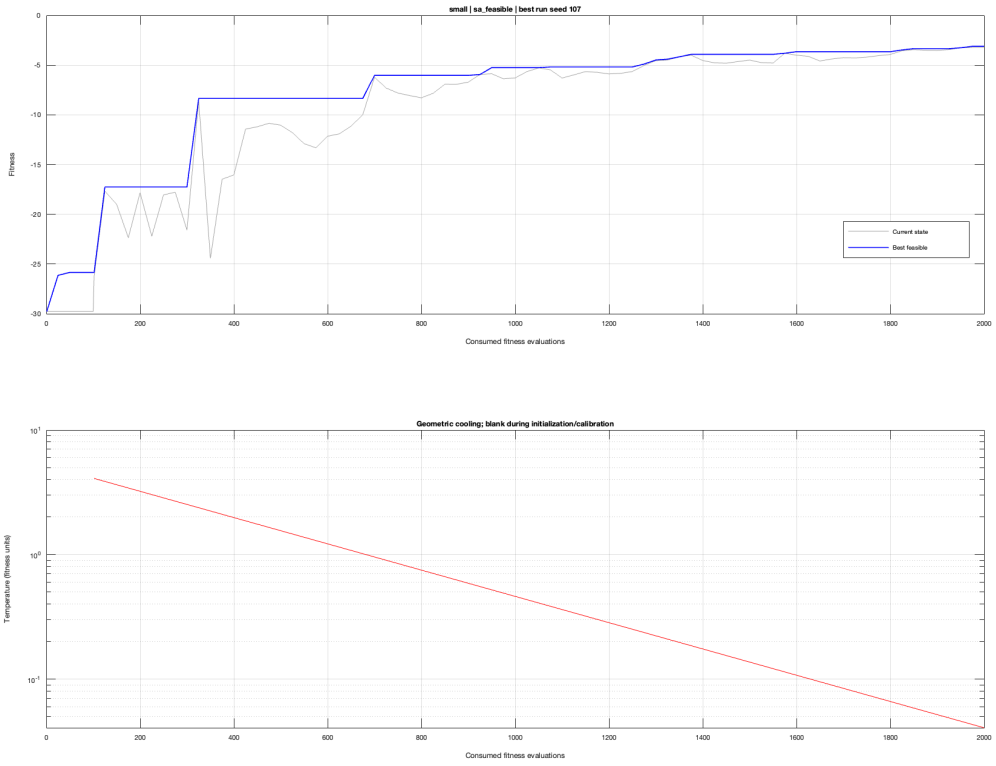


Figure B.3: Small case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot.

B.2 Large case

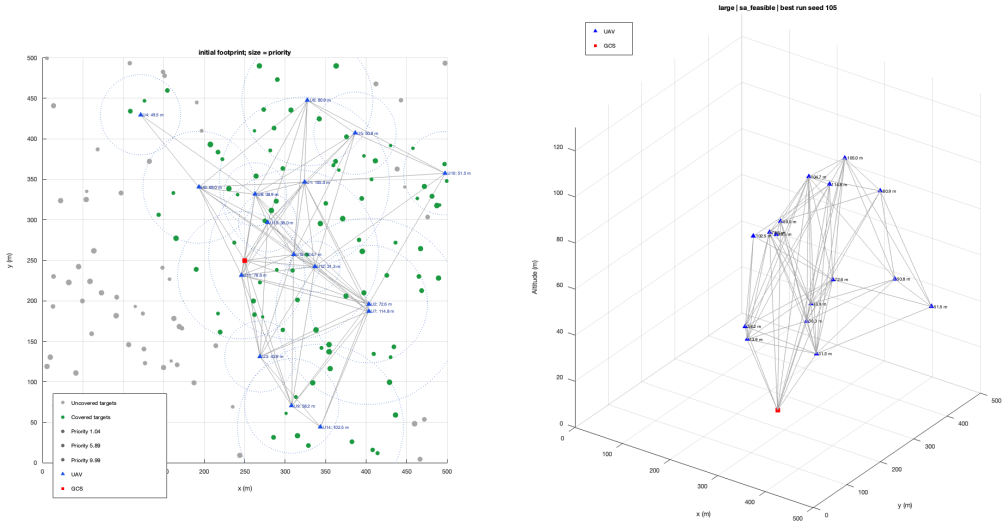


Figure B.4: Large case: original feasible placement of the best SA run, with its seed identified in the supplied plot.

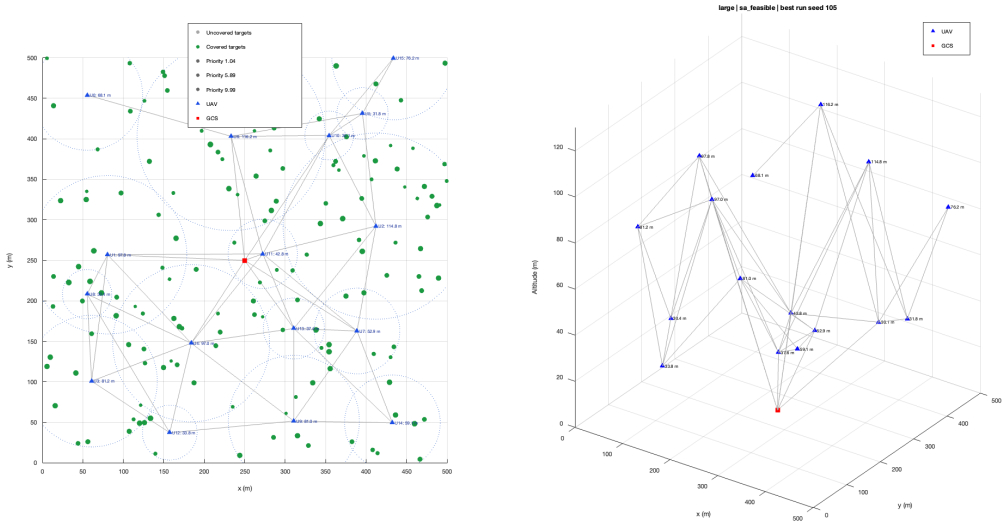


Figure B.5: Large case: best feasible placement of the best SA run, with its seed identified in the supplied plot.

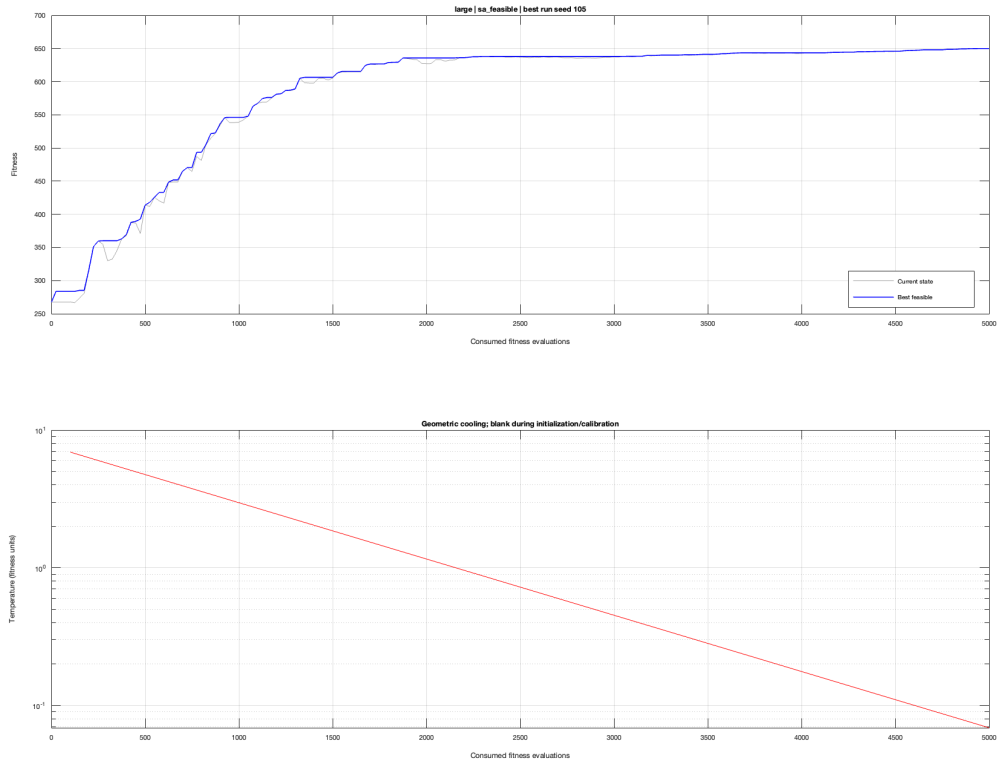


Figure B.6: Large case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot.

B.3 Clustered case

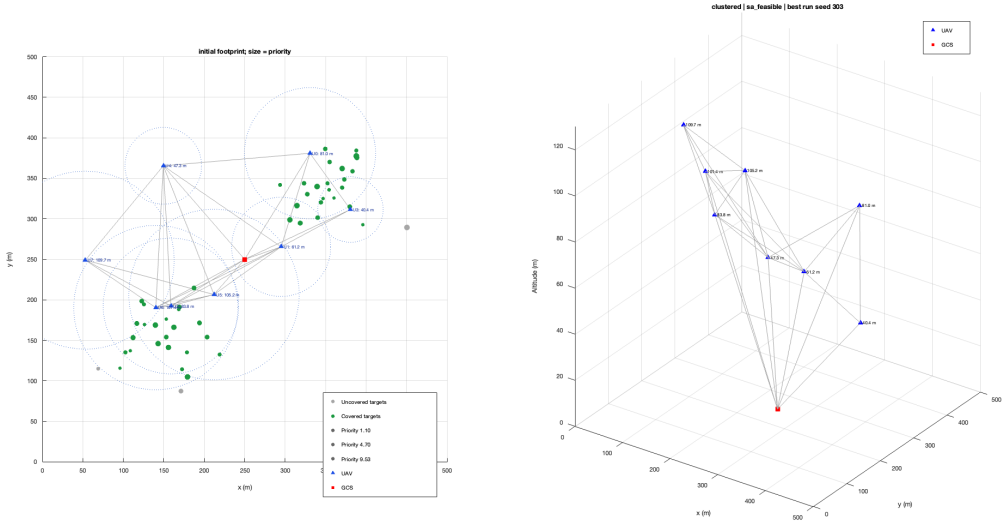


Figure B.7: Clustered case: original feasible placement of the best SA run, with its seed identified in the supplied plot.

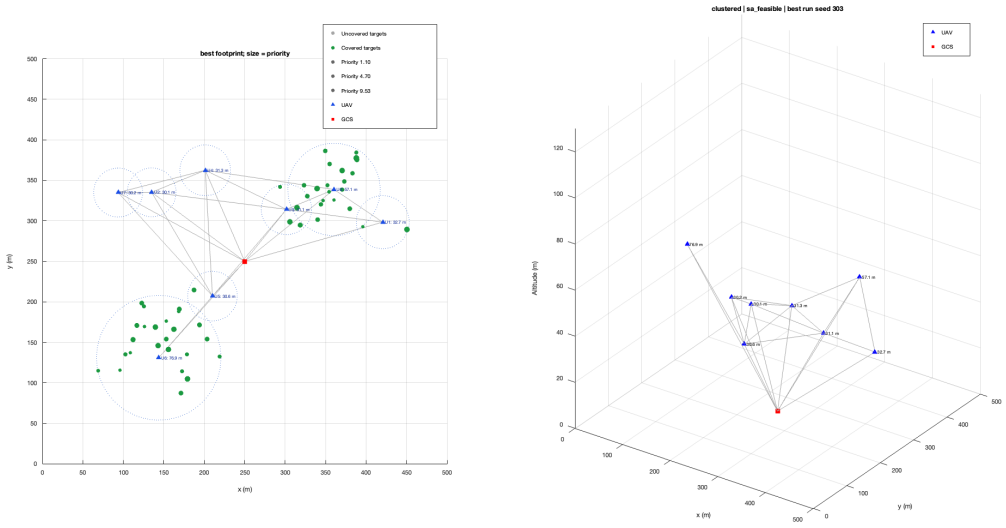


Figure B.8: Clustered case: best feasible placement of the best SA run, with its seed identified in the supplied plot.

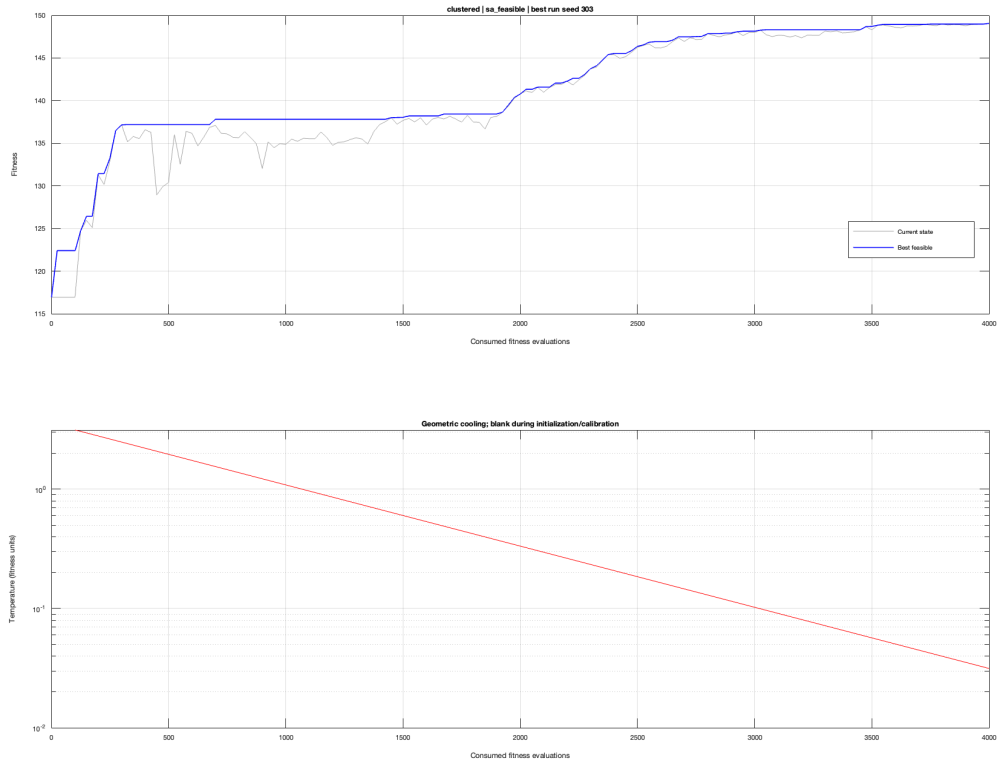


Figure B.9: Clustered case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot.

B.4 Separated case

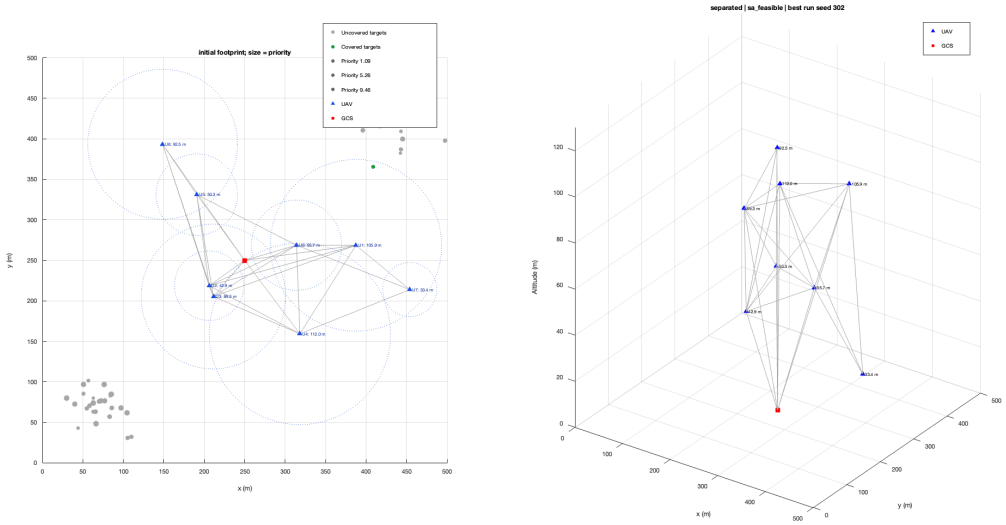


Figure B.10: Separated case: original feasible placement of the best SA run, with its seed identified in the supplied plot.

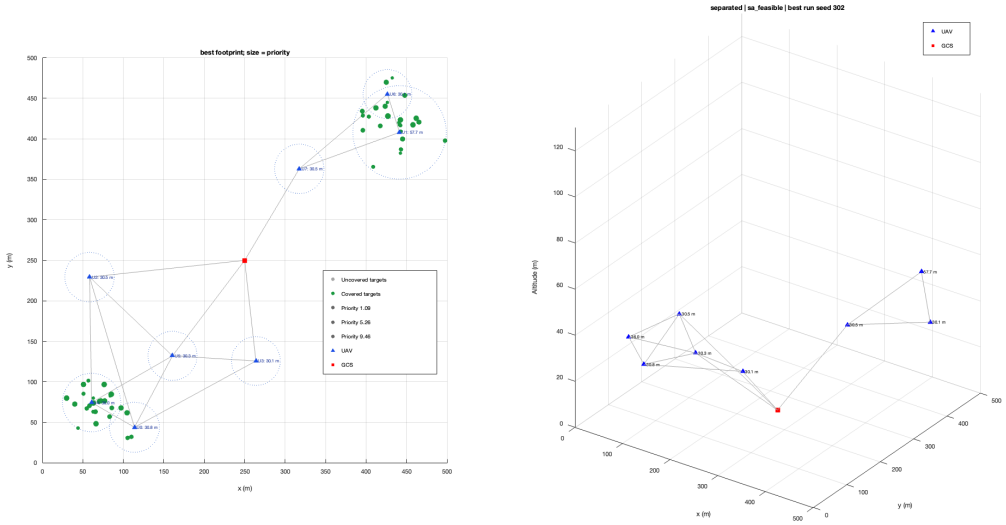


Figure B.11: Separated case: best feasible placement of the best SA run, with its seed identified in the supplied plot.

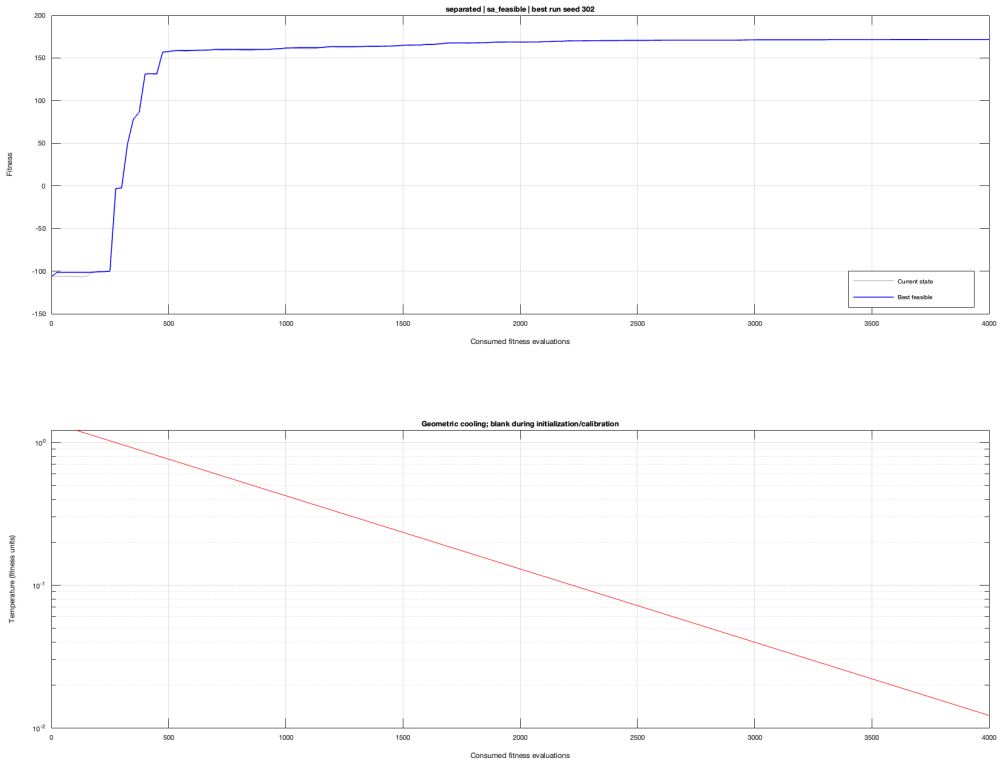


Figure B.12: Separated case: current/best fitness and temperature of the best SA run, with its seed identified in the supplied plot.

References

[1] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2334–2360, 2019. doi:10.1109/COMST.2019.2902862.

[2] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983. doi:10.1126/science.220.4598.671.